



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА - Российский технологический университет»

РТУ МИРЭА

Практическое занятие №7

ДИСЦИПЛИНА

Веб-программирование

(полное наименование дисциплины)

ИНСТИТУТ

Перспективных технологий и индустриального
программирования

КАФЕДРА

Компьютерного дизайна

(полное наименование кафедры)

ВИД УЧЕБНОГО
МАТЕРИАЛА

Практическое занятие

(в соответствии с пп.1-11)

ПРЕПОДАВАТЕЛЬ

Горбачев Сергей Геннадьевич

(фамилия, имя, отчество)

СЕМЕСТР

6, 2025/26

(указать семестр обучения, учебный год)

Москва 2026

Занятие 7

Общие замечания. Что уже есть.

1) у нас есть массив с объектами. Данные вводятся в файле php. Должно быть три явных поля — имя, адрес электронной почты, телефон.

Подробнее

- вводятся данные.
- из данных делаются объекты.
- объекты добавляются в массив.
- массив выводится на экран в виде таблицы. Это реализовано в виде функции. На входе функции — массив объектов. Обход массива делается с помощью foreach.

1) Вспомним.

- В качестве индексов массивов могут выступать не только числа, но и строки. В последнем случае массив называют **ассоциативным**, а индексы - его ключами. Если в качестве индексов массива выступают числа, он называется **индексным**.

- Вот так можно в переборе положить массив в переменные.

```
foreach ( $ship as $ key => list($fst, $snd, $thd) ) {  
    echo " <b>$ key</b><br />" ;  
    echo " < l i>$ fst</li>" ;  
    echo " < l i>$s nd</li>" ;  
    echo " < l i>$thd< / l i> " ;  
}
```

Или так -

```
foreach ( $ship as $ key => [$fst, $snd, $thd] ) {  
    echo " <b>$ key</b><br />" ;  
    echo " < l i>$ fst</li>" ;  
    echo " < l i>$s nd</li>" ;  
    echo " < l i>$thd< / l i> " ;  
}
```

- Вспомним **передачу параметров в функцию по значению и по ссылке**.

```
function getSum($var) // аргумент передается по значению  
{  
    $var = $var + 5 ;  
    return $var;  
}
```

```
$new_var = 20;  
echo getSum($new_var) ; // 25  
echo " <br />$new_var" ; // 20
```

```
function getSum(&$var) // аргумент передается по ссылке
{
$var = $var + 5 ;
return $var;
}
```

```
$new_var = 20;
echo getSum($new_var) ; // 25
echo " <br />$new_var" ; // 25
```

2) Сечение массива — часть массивам

array_slice - она возвращает часть массива \$array, начиная с элемента со смещением \$offset от начала.

```
$arr = [ 'a' , 'b' , 'c' , 'd' , 'e' ] ;
```

```
$out = array_slice($arr, 2) - возвращает 'c' 'd' 'e'
```

```
$out = array_slice($arr, 0, 3) - возвращает 'a' 'b' , 'c'
```

Вопрос — что будет в итоге с массивом \$arr — он изменится или нет?

Функция array_splice удаляет часть массива или заменяет ее частью другого массива .

```
$arr = [ 'красный' , 'зеленый' , 'синий' , 'желтый' ] ;
$out = array_splice($arr , 2) ; // [ 'красный' , 'зеленый' ]
```

Вопрос — что будет в итоге с массивом \$arr — он изменится или нет?

3) Слияние массивов - оператор «плюс» (+) , позволяющий складывать значения двух переменных, применим и для массивов.

Если в обоих складываемых массивах присутствуют элементы с одинаковыми индексами, в результирующий массив попадают элементы из левого массива.

4) Для того чтобы в результирующий массив попали элементы обоих массивов, вместо оператора + используют специальную функцию **array_merge(), которая имеет следующий синтаксис.**

```
$sum = array_merge($fst, $snd) ;
```

5) Сравнение массивов.

Как и любые две переменные, массивы можно сравнивать при помощи операторов равенства `==` и неравенства `!=`. Два массива считаются равными, если количество и значения их ключей и значений совпадают.

Вопрос — а что такое `===` и `!==` ?

6) Функция **`array_diff ( )`** определяет **исключительное пересечение** массивов

7) Функция **`array _ intersect( )`** определяет **включительное пересечение** массивов, т. е. возвращает массив, который содержит значения массива `$array`, имеющиеся во всех остальных массивах.

8) Функция **`in_ array()`** осуществляет поиск значения в массиве.

```
$userNameArray = [];  
$userNameArray[] = "Sergey";  
$userNameArray[] = "Olga";  
$userNameArray[] = "Anna";  
  
if(in_array('Sergey', $userNameArray)){  
    echo "yes\n";  
}  
else {  
    echo "no\n";  
}
```

Написать функцию поиска в нашем массиве объектов.

9) Для подсчета количества элементов в массиве предназначена функция **`count ( )`**

10) Найти ключ массива по значению позволяет функция следующий синтаксис:
`array_search()`

```
$array = [0 => ' blue ', 1 => ' red ', 2 => ' green ', 3 => ' red ' ] ;  
echo array_search ( ' green ', $array ) ;
```

11) Получение случайного значения обеспечивает функция `array_rand`, которая возвращает массив выбранных случайным образом индексов элементов массива.

```
print_r($userArray[array_rand($userArray)]);
```

12) Еще одной полезной функцией для получения случайных значений является функция `shuffle`, которая перемешивает элементы массива случайным образом.

13) Разные функции сортировки массива.

Функция `sort ( )` позволяет сортировать массив `$array` по возрастанию

Функция `rsort ( )` сортировки массива по убыванию

Функция `asort ( )` осуществляет сортировку ассоциативного массива по возрастанию.

Функция `arsort ( )` аналогична функции `asort ( )`, только она упорядочивает массив не по возрастанию, а по убыванию.

Функция `krsort ( )` осуществляет сортировку не по значениям, а по ключам массива в порядке их возрастания, при этом связи между ключами и значениями сохраняются.

Функция `ksort ( )` осуществляет сортировку элементов массива по ключам в обратном порядке (по убыванию):

Функция `array_multisort ( )` может быть использована для сортировки сразу нескольких массивов или одного многомерного массива

Задание — написать собственную функцию сортировки массива по имени

14) Функция `array_push ( )` добавляет к массиву новые элементы

Функция `array_pop ( )` извлекает элемент с вершины стека (последний элемент массива), удаляет его и возвращает массив без удаленного элемента